KPR INSTITUTE OF ENGINEERING AND TECHNOLOGY

Department of Artificial Intelligence and Data Science

PROJECT REPORT

A Project Report on

Breast Cancer Diagnosis from Cell-Nucleus Features Using a Deep Learning Multi-Layer Perceptron with Seven-Stage Data Visualization

Submitted by:

Student Name : Sredivit K T

Roll No. (Register Number) : 24AD112

3rd Year, B.Tech (AI & DS)

KPR Institute of Engineering and Technology
Learn Beyond (Autonomous, NAAC "A")
Avinashi Road, Arasur, Coimbatore.

Academic Year 2026 – 2027

Date of Submission : ____ / ____ / 2026

TABLE OF CONTENTS

1.	TITLE PAGE	1
2.	ABSTRACT	3
3.	INTRODUCTION & OBJECTIVE	3
4.	DATASET DESCRIPTION	4
5.	EDA AND PREPROCESSING	5
6.	DATA VISUALIZATION	6
7.	DEEP LEARNING MODEL	10
8.	RESULT VISUALIZATION & INTERPRETATION	12
9.	CONCLUSION AND FUTURE SCOPE	14
10.	REFERENCES	15
11.	APPENDIX (CODE SECTION)	17

2. Abstract

Early and accurate classification of breast tumours as benign or malignant supports timely treatment. This project builds a deep learning classifier on the Breast Cancer Wisconsin (Diagnostic) dataset, which has 569 samples and 30 numeric features computed from digitized fine-needle-aspirate images. The data had no missing values or duplicates; extreme outliers were capped, features standardized, and the data split 70/15/15 with stratification. Seven visualizations, following Fry's seven stages, showed that size and concavity features separate the classes best and that many features are highly correlated. A Keras multi-layer perceptron (32-16-1) was tuned over a 12-point grid and evaluated on a held-out test set, reaching 98.8% accuracy, 0.969 recall and an AUC of 0.9994, against 96.5% accuracy for a logistic regression baseline.

Keywords: breast cancer, deep learning, multi-layer perceptron, data visualization, binary classification.

3. Introduction & Objective

3.1 Introduction

Breast cancer is one of the most common cancers worldwide, and the outcome for a patient depends strongly on how early the disease is found and correctly classified. In fine-needle aspiration (FNA), a thin needle takes a small sample of cells from a breast mass. The sample is digitized and the cell nuclei are measured. The measurements describe the size and shape of the nuclei, which differ between benign and malignant tumours.

The problem addressed here is to predict, from 30 numeric nucleus measurements, whether a tumour is **benign** or **malignant**. This is a binary classification problem on tabular data. Because a missed cancer is more harmful than a false alarm, recall for the malignant class is monitored along with accuracy.

3.2 Objectives

- To select a real-world dataset with at least 15 features and understand it through exploratory data analysis (EDA).
- To clean the data (missing values, duplicates, outliers), scale the features and create train, validation and test sets.
- To create at least five meaningful visualizations covering all seven stages of visualization, and to explain what each shows.
- To build a deep learning model (multi-layer perceptron), analyse feature importance and tune basic hyperparameters.
- To evaluate the model with loss and accuracy curves, a confusion matrix, an ROC curve with AUC and an error-distribution chart.

- To summarize the findings, the challenges faced and the scope for future improvement.

4. Dataset Description

4.1 Source

The dataset is the **Breast Cancer Wisconsin (Diagnostic)** dataset from the UCI Machine Learning Repository, created by Wolberg, Street and Mangasarian [1]. A copy is distributed with scikit-learn (load_breast_cancer), which was used to load it so that the notebook runs without a separate download. The features were computed from digitized images of FNA samples, as described in [2].

4.2 Size and Fields

The dataset has **569 samples** (rows) and **30 numeric features**, plus a binary target. There are **357 benign (62.7%)** and **212 malignant (37.3%)** cases. For each of ten cell-nucleus properties, three values are stored: the **mean**, the **standard error** and the **worst** value (the mean of the three largest values). This gives 10 x 3 = 30 features.

Table 4.1: The ten base properties of the cell nucleus

S.No	Property	Meaning
1	Radius	Mean distance from the centre to points on the boundary
2	Texture	Standard deviation of grey-scale values
3	Perimeter	Length of the nucleus boundary
4	Area	Area enclosed by the nucleus
5	Smoothness	Local variation in radius lengths
6	Compactness	Perimeter squared divided by area, minus 1
7	Concavity	Severity of concave portions of the contour
8	Concave points	Number of concave portions of the contour
9	Symmetry	Symmetry of the nucleus shape
10	Fractal dimension	Coastline approximation minus 1 (boundary roughness)

4.3 Basic Statistics

Table 4.2 gives summary statistics for seven representative features, together with the class-wise means. Malignant tumours have clearly larger and more irregular nuclei: for example, the mean of worst perimeter is about 141 for malignant and 87 for benign cases.

Table 4.2: Summary statistics of representative features

Feature	Mean	Std	Min	Max	Benign mean	Malignant mean
mean radius	14.127	3.524	6.981	28.11	12.147	17.463
mean texture	19.29	4.301	9.71	39.28	17.915	21.605
mean area	654.889	351.914	143.5	2501	462.79	978.376
mean smoothness	0.096	0.014	0.053	0.163	0.092	0.103
mean concavity	0.089	0.08	0.000	0.427	0.046	0.161
worst perimeter	107.261	33.603	50.41	251.2	87.006	141.37
worst concave points	0.115	0.066	0.000	0.291	0.074	0.182

5. EDA and Preprocessing

5.1 Methods Used

The EDA used summary statistics, data types, class counts, missing-value and duplicate checks, the IQR rule for outliers, and correlation analysis. Preprocessing was written so that every statistic that is learned from data (median, outlier bounds, mean and standard deviation) is computed on the **training set only**, which avoids data leakage into the validation and test sets.

Missing values and duplicates. There were 0 missing values and 0 duplicate rows. A median imputer was still included in the pipeline so that it is safe for future data.

Outliers. The 1.5 x IQR rule flagged 608 of 17,070 cells (3.6%), and 171 of the 569 rows had at least one flagged value. Table 5.1 lists the features with the most flagged values. Outliers appear in both classes, and many are real tumour variation (for example very large malignant tumours) rather than recording errors, so removing rows would discard useful cases. Instead, values beyond **3 x IQR** were capped (winsorized) using bounds learned from the training set; this changed 86 values in the training data.

Table 5.1: Features with the most IQR outliers (1.5 x IQR rule)

S.No	Feature	Outliers flagged
1	area error	65
2	radius error	38
3	perimeter error	38
4	worst area	35
5	smoothness error	30

Encoding, scaling and splitting. The target was encoded as 1 for malignant (the positive class) and 0 for benign. All 30 features are numeric, so no categorical encoding was needed. Features were **standardized** (zero mean, unit variance) with a scaler fitted on the training set. The data was

split into train, validation and test sets using stratified sampling so that each set keeps the same class ratio (Table 5.2).

Table 5.2: Train, validation and test split

Set	Samples	Share	Malignant share
Train	398	70%	37.2%
Validation	85	15%	37.6%
Test	86	15%	37.2%

5.2 Insights

- The data is clean, and the classes are mildly imbalanced (62.7% benign), so accuracy alone is not enough; recall, precision, F1 and AUC are also reported.
- Size and shape features (radius, perimeter, area, concavity, concave points) differ strongly between classes.
- Features are highly redundant. Radius, perimeter and area correlate at 0.98 to 0.998, and 21 feature pairs have |r| above 0.9. Effective dimensionality is much lower than 30.
- Several "error" features are strongly right-skewed (for example area error, skewness 5.45), which supports scaling and conservative outlier handling.

6. Data Visualization

Seven visualizations were created, one for each of Ben Fry's seven stages of visualization [6]: Acquire, Parse, Filter, Mine, Represent, Refine and Interact. Figures 6.1 to 6.6 are shown below; the seventh is an interactive Plotly chart that is saved as an HTML file in the project repository.

6.1 V1: Class Distribution (Stage 1: Acquire)

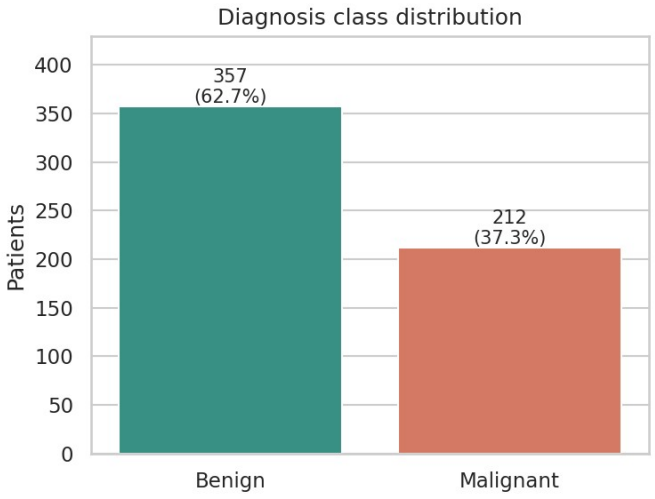


Fig. 6.1: Class distribution of the diagnosis

What it shows: The number of benign and malignant samples.

Why it was created: Class balance is the first thing to check after acquiring data, because it decides which metrics can be trusted.

Insights: There are 357 benign (62.7%) and 212 malignant (37.3%) cases. The imbalance is mild. Predicting "benign" for every case would already give 62.7% accuracy, so recall, precision, F1 and AUC are used as well, together with stratified splits.

6.2 V2: Feature Histograms by Class (Stage 2: Parse)

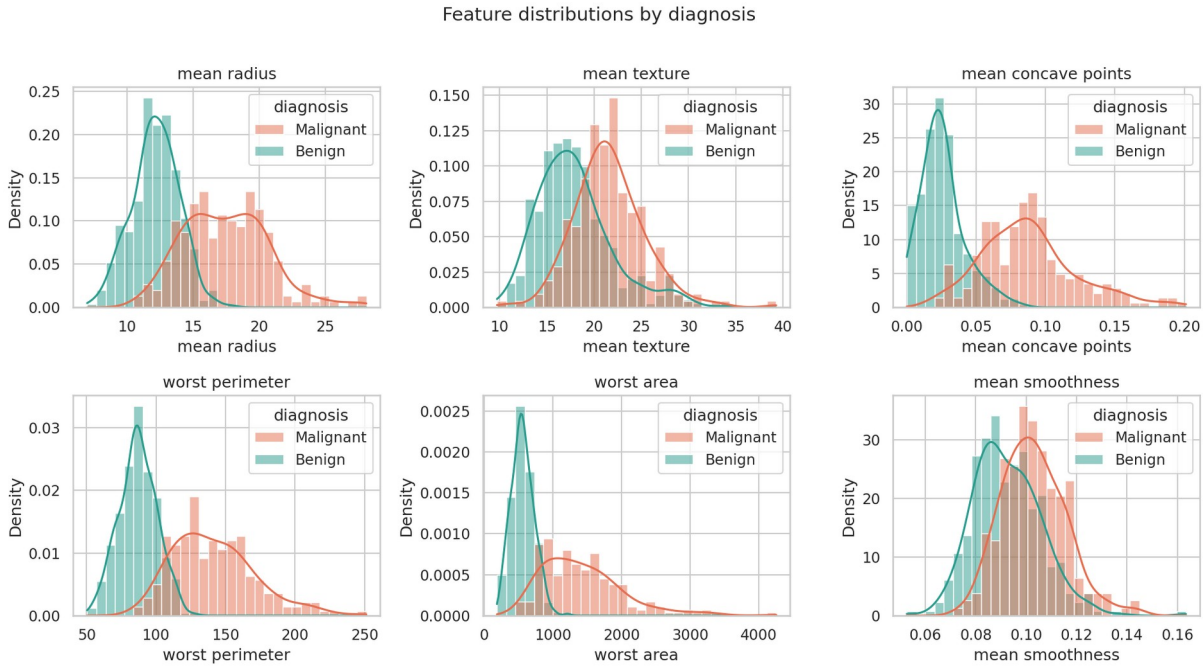


Fig. 6.2: Distributions of six features by diagnosis

What it shows: The distribution of six representative features, drawn separately for each class.

Why it was created: After the columns are parsed into numeric fields, the shape, skew and overlap between classes of each feature must be seen.

Insights: Radius, perimeter, area and concave points shift clearly to higher values for malignant tumours. Area is right-skewed, which supports scaling. Mean smoothness and mean texture overlap strongly, so they separate the classes less well on their own.

6.3 V3: Boxplots (Stage 3: Filter)

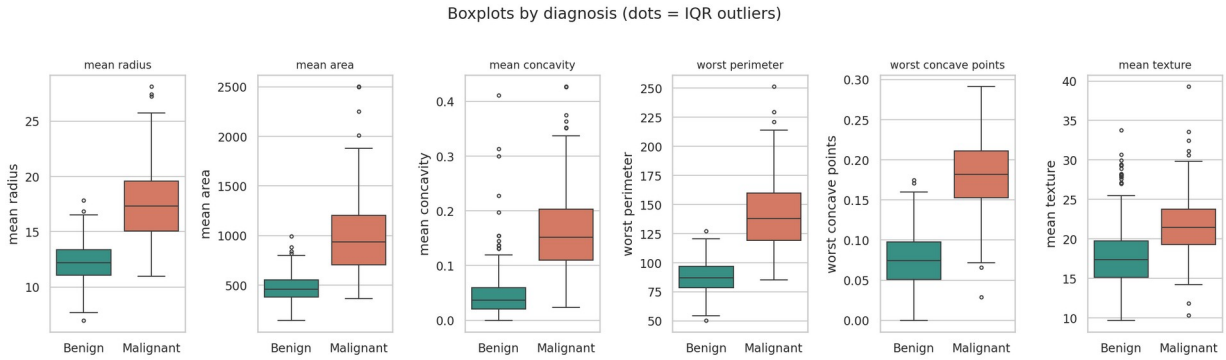


Fig. 6.3: Boxplots of six features by diagnosis

What it shows: The median, spread and outliers of key features for each class.

Why it was created: This is the filtering step: it shows which values are unusual and which features carry signal, before deciding how to treat outliers.

Insights: Medians differ strongly between the classes for radius, area, concavity and the worst-case features, while mean texture separates them only moderately. Outliers appear in both classes: malignant cases at the high end of size features, and benign cases mainly in concavity and texture. Many are real tumour variation, so only extreme values were capped instead of deleting rows.

6.4 V4: Correlation Heatmap (Stage 4: Mine)

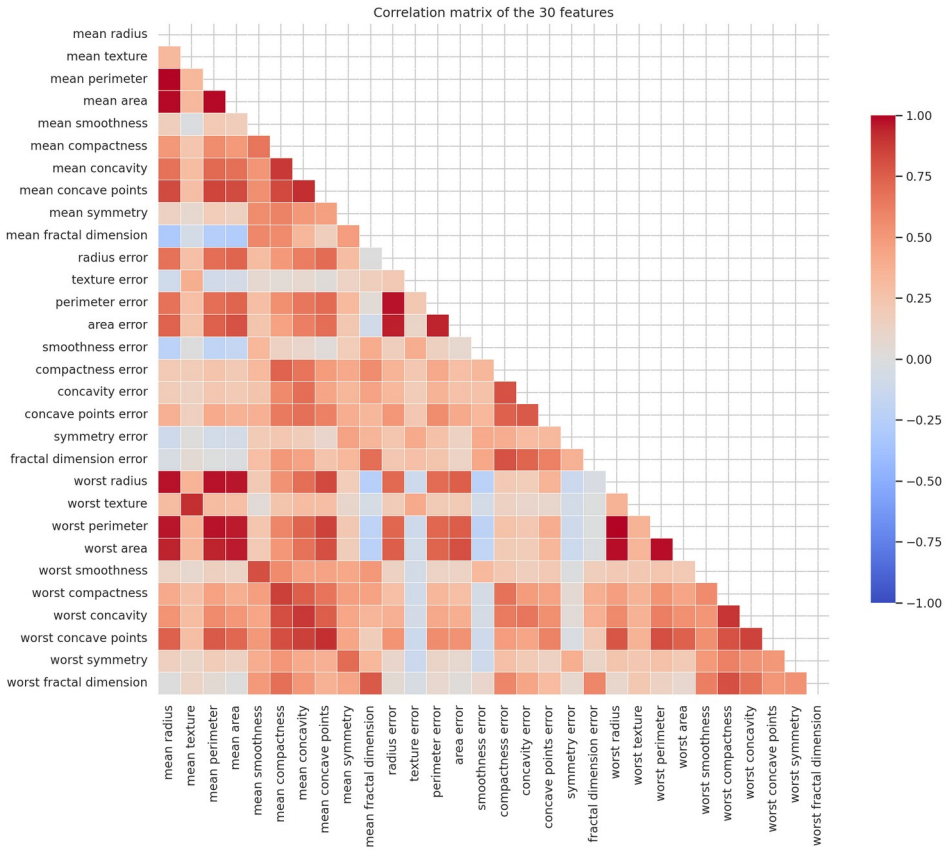


Fig. 6.4: Correlation matrix of the 30 features

What it shows: The pairwise Pearson correlation between all 30 features.

Why it was created: To mine the data for structure, in particular redundant features.

Insights: Radius, perimeter and area are almost perfectly correlated (mean radius and mean perimeter r = 0.998), each "mean" feature is strongly tied to its "worst" version (r between 0.81 and 0.97), and 21 pairs have |r| above 0.9. This multicollinearity means the data is effectively low-dimensional, and it also spreads importance scores across related features.

6.5 V5: Pairplot (Stage 5: Represent)

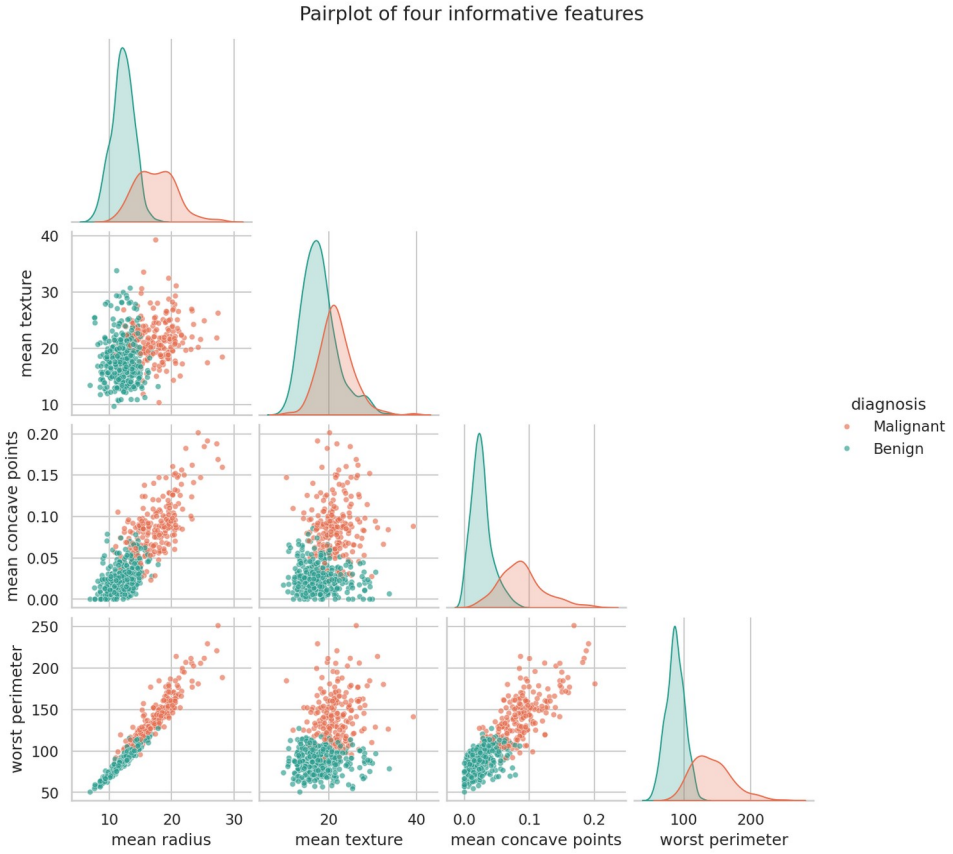


Fig. 6.5: Pairplot of four informative features

What it shows: All pairwise scatterplots and the per-class density of four features: mean radius, mean texture, mean concave points and worst perimeter.

Why it was created: A scatter matrix is a suitable representation to judge class separability from many angles at once.

Insights: The classes form two largely distinct clusters in the size and concavity planes, so a non-linear model should separate them well. A small overlap zone near the boundary is where mistakes are most likely. Mean texture adds little separation.

6.6 V6: Refined Scatterplot (Stage 6: Refine)

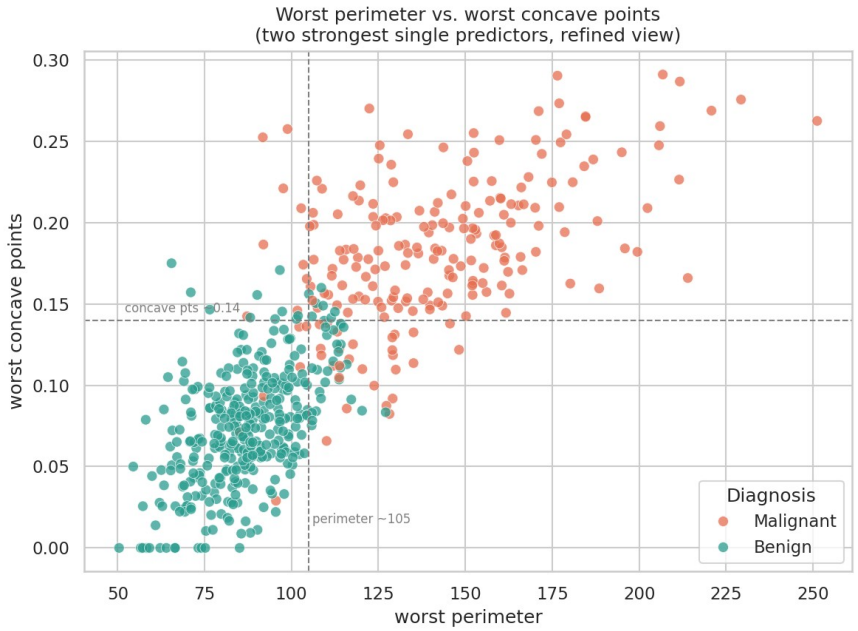


Fig. 6.6: Worst perimeter versus worst concave points

What it shows: The two features that correlate most strongly with the target (|r| about 0.78), with approximate guide lines.

Why it was created: Refinement removes clutter and adds reference lines so the structure is clear.

Insights: Benign cases gather at the bottom-left and malignant cases at the top-right, with a thin mixed band between them. The guide lines are approximate and were chosen by eye, not fitted. A few simple cut-offs already separate most cases, which is consistent with a linear baseline performing well later.

6.7 V7: Interactive Scatter (Stage 7: Interact)

The same relationship as V6 is provided as an interactive Plotly chart (figures/v7_interactive_scatter.html in the repository). Marker size shows mean area, hovering shows extra measurements, and clicking the legend filters by class. **Why it was created:** the interact stage lets an analyst zoom into the borderline region and inspect individual cases. **Insights:** the region where the two colours mix is where extra features or a second opinion would help most.

7. Deep Learning Model

7.1 Architecture

A **multi-layer perceptron (MLP)** was chosen because the data is tabular with a modest number of features. The network has two hidden layers. Each hidden layer is followed by batch normalization and dropout, and the output is a single sigmoid unit that gives the probability that the tumour is

malignant. L2 regularization (1e-4) is applied to the hidden-layer weights. The total is 1,729 parameters (1,633 trainable).

Table 7.1: Model architecture

No.	Layer	Output shape	Parameters
1	Input (standardized features)	(None, 30)	0
2	Dense, 32 units, ReLU	(None, 32)	992
3	Batch Normalization	(None, 32)	128
4	Dropout (0.1)	(None, 32)	0
5	Dense, 16 units, ReLU	(None, 16)	528
6	Batch Normalization	(None, 16)	64
7	Dropout (0.1)	(None, 16)	0
8	Dense, 1 unit, Sigmoid	(None, 1)	17
	Total		**1,729**

7.2 Training Parameters

Table 7.2: Training settings

Setting	Value
Loss function	Binary cross-entropy
Optimizer	Adam, learning rate 0.005
Batch size	32
Maximum epochs	150
Early stopping	Monitor validation loss, patience 15, restore best weights
Epochs actually run	38 (best epoch: 23)
Regularization	L2 (1e-4) and dropout (0.1)
Metrics tracked	Accuracy and AUC
Random seed	42
Data used	Train 398 rows for fitting; validation 85 rows for stopping and tuning

7.3 Hyperparameters

A basic grid search was run over the number of units in the first hidden layer (16, 32, 64), the dropout rate (0.1, 0.3) and the learning rate (0.001, 0.005), giving 12 models. Each model was trained with early stopping and ranked by **validation loss**; the test set was not used. The best configuration was **32 units, dropout 0.1 and learning rate 0.005**. The differences among the top configurations are less than one validation sample (1.2 percentage points), so the choice is not strongly separated.

Table 7.3: Hyperparameter search results (sorted by validation loss)

Units	Dropout	Learn. rate	Val. loss	Val. acc.	Val. AUC	Best epoch
32	**0.1**	**0.005**	**0.0737**	**0.9882**	**0.9976**	**23**
64	0.1	0.005	0.0759	0.9882	0.9971	14
32	0.3	0.005	0.0781	0.9882	0.9968	26
64	0.1	0.001	0.0799	0.9765	0.9976	32
64	0.3	0.005	0.0823	0.9765	0.9971	16
64	0.3	0.001	0.0835	0.9765	0.9976	37
32	0.3	0.001	0.085	0.9765	0.9959	53
16	0.3	0.001	0.087	0.9882	0.9903	65
16	0.3	0.005	0.0889	0.9882	0.9826	29
32	0.1	0.001	0.0895	0.9647	0.9956	57
16	0.1	0.001	0.0933	0.9765	0.9891	44
16	0.1	0.005	0.1136	0.9765	0.9829	33

8. Result Visualization & Interpretation

All results in this section are measured on the **held-out test set of 86 samples** (54 benign, 32 malignant) that was not used for tuning.

8.1 Accuracy, Precision, Recall and AUC

Table 8.1: Test-set performance of the MLP and a logistic regression baseline

Model	Accuracy	Precision	Recall	AUC
MLP (Keras)	**0.9884**	**1.0000**	**0.9688**	**0.9994**
Logistic Regression	0.9651	0.9677	0.9375	0.9988

Table 8.2: Per-class report of the MLP (test set)

Class	Precision	Recall	F1-score	Support
Benign	0.982	1.000	0.991	54
Malignant	1.000	0.969	0.984	32
Overall accuracy			0.988	86

The MLP reached 98.8% accuracy (85 of 86 correct), precision of 1.00 and recall of 0.969 for malignant cases. With only 86 test samples, the 95% Wilson confidence interval for the accuracy is about 93.7% to 99.8%, so the result should be read as a range, not an exact figure.

8.2 Loss and Accuracy versus Epoch

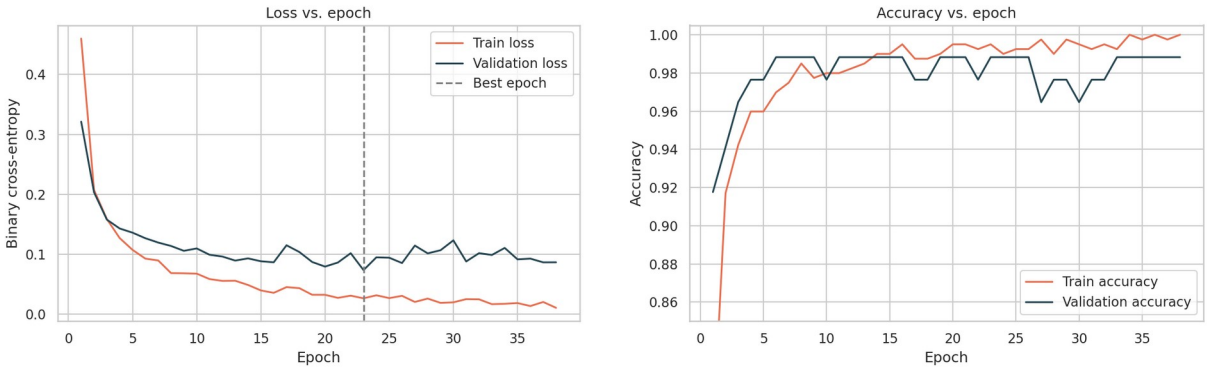


Fig. 8.1: Loss versus epoch (left) and accuracy versus epoch (right)

Loss: both curves fall quickly in the first five epochs. After that, training loss keeps decreasing to about 0.01 while validation loss levels off near 0.08 to 0.10 and fluctuates, so a gap opens, which indicates **mild overfitting** (expected with about 1,700 parameters and 398 training rows). Early stopping at epoch 23, the lowest validation loss (dashed line), restored the best weights and limited the damage.

Accuracy: accuracy passes 96% within about five epochs. Training accuracy approaches 100%, while validation accuracy stays around 97% to 99%, which agrees with the gap seen in the loss chart. Because the validation set has 85 samples, each sample is worth about 1.2 percentage points, so the validation curve moves in steps.

8.3 Confusion Matrix and ROC Curve

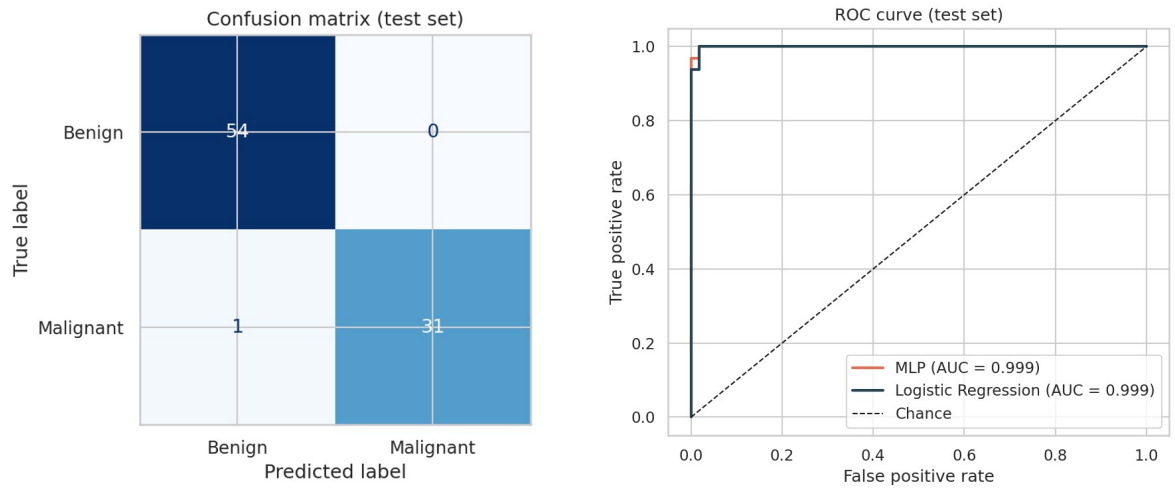


Fig. 8.2: Confusion matrix (left) and ROC curve with AUC (right) on the test set

Confusion matrix: the model produced **0 false positives and 1 false negative** (TN = 54, FP = 0, FN = 1, TP = 31). A false negative, a malignant tumour called benign, is clinically more serious than a false positive, which only leads to extra testing. In a screening setting the decision threshold could be lowered to raise recall, at the cost of a few more false alarms.

ROC curve and AUC: the curve hugs the top-left corner, meaning high recall at a very low false-positive rate. The MLP has an AUC of 0.9994 and the logistic regression 0.9988, so both rank cases almost equally well, which fits the near-linear separability seen in the visualizations. The accuracy gap (98.8% versus 96.5%) is only 2 test samples out of 86, so it should not be read as a reliable win for the deep model.

8.4 Error Distribution

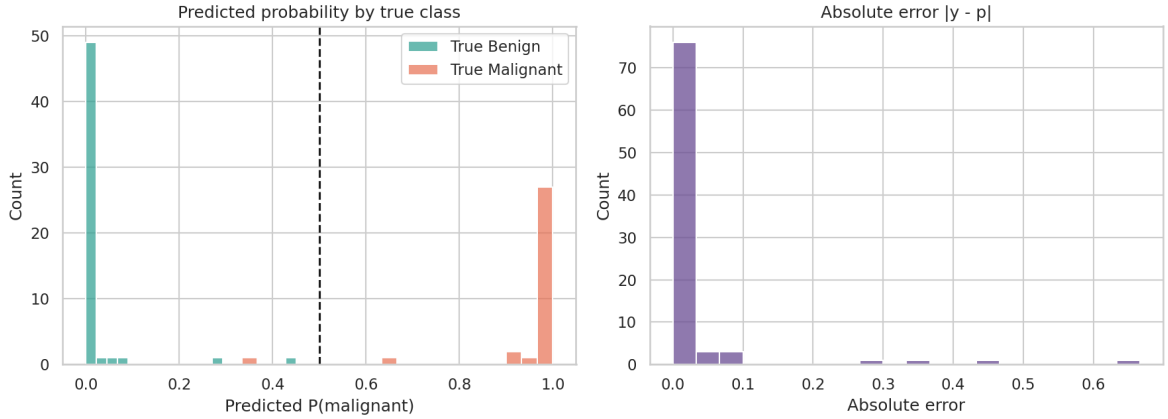


Fig. 8.3: Predicted probability by true class (left) and absolute error (right)

Predicted probabilities pile up near 0 and 1, so the model is confident and mostly correct. The mean absolute error is 0.027, and only one test sample has an error above 0.5: a malignant tumour given a malignant probability of about 0.35, which is the single false negative. A few other cases lie between 0.3 and 0.65 but on the correct side of the threshold. These are the borderline cases anticipated in V6 and V7.

9. Conclusion and Future Scope

9.1 Conclusion

The dataset is clean (no missing values or duplicates) and mildly imbalanced (62.7% benign). Size and concavity measurements, especially the worst-case values (perimeter, radius, area, concave points), separate malignant from benign tumours best, and the 30 features are highly redundant, so the information is effectively low-dimensional.

A small MLP (32-16-1 with batch normalization, dropout 0.1, L2 and Adam at learning rate 0.005) reached **98.8% test accuracy, precision 1.00, recall 0.969, F1 0.984 and AUC 0.9994**, with 1 false negative and 0 false positives out of 86 test samples. The logistic regression baseline scored 96.5% accuracy and AUC 0.9988. The deep model is slightly ahead on accuracy but tied on AUC; with 86 test samples this difference is within noise, which suggests that the problem is close to linearly separable and that depth adds limited value here. These are single-split results.

The main challenges were:

- **Small data:** the test set has only 86 samples (validation 85), so one error changes accuracy by about 1.2 points.
- **Outliers versus signal:** large values are often genuine malignant cases, so conservative capping was used instead of removal.
- **Mild overfitting:** training loss kept falling while validation loss levelled off; early stopping and dropout limited it but did not remove it.
- **Unequal cost of errors:** a missed cancer is worse than a false alarm, so accuracy alone is not the right target.

9.2 Future Scope

- Use repeated stratified k-fold cross-validation and report confidence intervals.
- Tune the decision threshold for a target recall (for example at least 99%) and calibrate the predicted probabilities.
- Use a wider search (Bayesian optimization or Keras Tuner) and compare with gradient boosting such as XGBoost or LightGBM.
- Add explainability methods such as SHAP or LIME to build clinical trust.
- Train a convolutional neural network directly on the cytology images instead of hand-crafted features, and add patient information such as age and family history.
- Validate on external hospital data to check for dataset shift. This project is educational and is not a diagnostic tool.

10. References

[1] W. H. Wolberg, W. N. Street, and O. L. Mangasarian, "Breast Cancer Wisconsin (Diagnostic)," UCI Machine Learning Repository, 1995. [Online]. Available: https://archive.ics.uci.edu/dataset/17

[2] W. N. Street, W. H. Wolberg, and O. L. Mangasarian, "Nuclear feature extraction for breast tumor diagnosis," in Proc. IS&T/SPIE Int. Symp. Electronic Imaging: Science and Technology, vol. 1905, 1993, pp. 861-870.

[3] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: A simple way to prevent neural networks from overfitting," J. Mach. Learn. Res., vol. 15, pp. 1929-1958, 2014.

[4] S. Ioffe and C. Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift," in Proc. 32nd Int. Conf. Machine Learning (ICML), 2015, pp. 448-456.

[5] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in Proc. Int. Conf. Learning Representations (ICLR), 2015.

[6] B. Fry, Visualizing Data: Exploring and Explaining Data with the Processing Environment. Sebastopol, CA, USA: O'Reilly Media, 2007.

[7] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," J. Mach. Learn. Res., vol. 12, pp. 2825-2830, 2011.

[8] M. Abadi et al., "TensorFlow: Large-scale machine learning on heterogeneous systems," 2015. [Online]. Available: https://www.tensorflow.org

11. Appendix (Code Section)

The complete Python code is given below. It covers data loading, EDA, preprocessing, visualization, model creation and training, hyperparameter tuning, and evaluation. The same code is available as a Jupyter notebook in the GitHub repository. **GitHub repository link:** _____

```
# ===== Step 1: Dataset Selection =====
import warnings, random, os
warnings.filterwarnings("ignore")
os.environ["TF_CPP_MIN_LOG_LEVEL"] = "3"
import numpy as np, pandas as pd
import matplotlib.pyplot as plt, seaborn as sns
import plotly.express as px
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.metrics import (confusion_matrix, ConfusionMatrixDisplay, roc_curve, auc,
                             classification_report, accuracy_score, precision_score,
                             recall_score, f1_score)
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.feature_selection import mutual_info_classif
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

SEED = 42
random.seed(SEED); np.random.seed(SEED); tf.random.set_seed(SEED)
sns.set_theme(style="whitegrid", context="notebook")
os.makedirs("figures", exist_ok=True)
PAL = {"Benign": "#2a9d8f", "Malignant": "#e76f51"}

raw = load_breast_cancer(as_frame=True)
df = raw.frame.copy()
# In sklearn: target 0 = malignant, 1 = benign. Make it explicit and readable.
df["diagnosis"] = df["target"].map({0: "Malignant", 1: "Benign"})
FEATURES = list(raw.feature_names)
print("Shape:", df.shape, "| features:", len(FEATURES))
df.head()

# ===== Step 2: Data Understanding and Preprocessing =====
print(df[FEATURES].dtypes.value_counts(), "\n")
print("Class counts:\n", df["diagnosis"].value_counts(), "\n")
print("Class share (%):\n",
  (df["diagnosis"].value_counts(normalize=True)*100).round(1))
df[FEATURES].describe().T.round(3).head(12)

# ===== 2.2 Missing values, duplicates, outliers =====
# Missing values
print("Total missing cells:", int(df[FEATURES].isna().sum().sum()))
# Duplicates
print("Duplicate rows:", int(df.duplicated(subset=FEATURES).sum()))

# Outliers: IQR rule, count per feature
Q1, Q3 = df[FEATURES].quantile(.25), df[FEATURES].quantile(.75)
```

```
IQR = Q3 - Q1
out_counts = ((df[FEATURES] < Q1 - 1.5*IQR) | (df[FEATURES] > Q3 +
  1.5*IQR)).sum().sort_values(ascending=False)
print("\nFeatures with most IQR outliers (1.5xIQR):")
print(out_counts.head(8))
print("Rows with >=1 outlier:", int(((df[FEATURES] < Q1 - 1.5*IQR) | (df[FEATURES] >
  Q3 + 1.5*IQR)).any(axis=1).sum()))

# ===== 2.3 Split, clean, and scale =====
X = df[FEATURES]
y = (df["diagnosis"] == "Malignant").astype(int)    # encode: 1 = Malignant (positive
  class), 0 = Benign

# 70 / 15 / 15 stratified split
X_tr, X_tmp, y_tr, y_tmp = train_test_split(X, y, test_size=0.30, stratify=y,
  random_state=SEED)
X_va, X_te, y_va, y_te = train_test_split(X_tmp, y_tmp, test_size=0.50,
  stratify=y_tmp, random_state=SEED)
print("Train/Val/Test:", len(X_tr), len(X_va), len(X_te))
print("Malignant share  ->", [round(v.mean(),3) for v in (y_tr, y_va, y_te)])

# Imputer + winsorize (train-only bounds) + standardize (train-only stats)
imp = SimpleImputer(strategy="median").fit(X_tr)
to_df = lambda a, idx: pd.DataFrame(a, columns=FEATURES, index=idx)
X_tr_i, X_va_i, X_te_i = to_df(imp.transform(X_tr), X_tr.index),
  to_df(imp.transform(X_va), X_va.index), to_df(imp.transform(X_te), X_te.index)

q1, q3 = X_tr_i.quantile(.25), X_tr_i.quantile(.75); iqr = q3 - q1
lo, hi = q1 - 3*iqr, q3 + 3*iqr
clip = lambda d: d.clip(lower=lo, upper=hi, axis=1)
X_tr_c, X_va_c, X_te_c = clip(X_tr_i), clip(X_va_i), clip(X_te_i)
print("Values clipped (train):", int((X_tr_i != X_tr_c).sum().sum()))

scaler = StandardScaler().fit(X_tr_c)
Xtr, Xva, Xte = (scaler.transform(d) for d in (X_tr_c, X_va_c, X_te_c))
ytr, yva, yte = y_tr.values, y_va.values, y_te.values
print("Train mean ~", Xtr.mean().round(3), "| std ~", Xtr.std().round(3))

# ===== V1: Class distribution (Stage 1: Acquire) =====
fig, ax = plt.subplots(figsize=(5.2,4))
order = ["Benign","Malignant"]
sns.countplot(data=df, x="diagnosis", order=order, palette=PAL, ax=ax)
for p in ax.patches:
    ax.annotate(f"{int(p.get_height())}\n({p.get_height()/len(df)*100:.1f}%)",
  (p.get_x()+p.get_width()/2, p.get_height()),
                ha="center", va="bottom", fontsize=10)
ax.set_ylim(0, 430); ax.set_title("Diagnosis class distribution"); ax.set_xlabel("");
  ax.set_ylabel("Patients")
plt.tight_layout(); plt.savefig("figures/v1_class_distribution.png", dpi=150);
  plt.show()

# ===== V2: Feature histograms by class (Stage 2: Parse) =====
show = ["mean radius","mean texture","mean concave points","worst perimeter","worst
  area","mean smoothness"]
fig, axes = plt.subplots(2,3, figsize=(13,7))
for ax, f in zip(axes.ravel(), show):
    sns.histplot(data=df, x=f, hue="diagnosis", palette=PAL, bins=30, kde=True,
  stat="density", common_norm=False, ax=ax)
```

```
    ax.set_title(f)
plt.suptitle("Feature distributions by diagnosis", y=1.01, fontsize=14)
plt.tight_layout(); plt.savefig("figures/v2_histograms.png", dpi=150,
  bbox_inches="tight"); plt.show()

# ===== V3: Boxplots for outlier inspection (Stage 3: Filter) =====
sel = ["mean radius","mean area","mean concavity","worst perimeter","worst concave
  points","mean texture"]
fig, axes = plt.subplots(1,6, figsize=(16,4.5))
for ax, f in zip(axes, sel):
    sns.boxplot(data=df, x="diagnosis", y=f, order=["Benign","Malignant"],
  palette=PAL, fliersize=3, ax=ax)
    ax.set_xlabel(""); ax.set_title(f, fontsize=10)
plt.suptitle("Boxplots by diagnosis (dots = IQR outliers)", y=1.03)
plt.tight_layout(); plt.savefig("figures/v3_boxplots.png", dpi=150,
  bbox_inches="tight"); plt.show()

# ===== V4: Correlation heatmap (Stage 4: Mine) =====
corr = df[FEATURES].corr()
mask = np.triu(np.ones_like(corr, dtype=bool))
fig, ax = plt.subplots(figsize=(13,11))
sns.heatmap(corr, mask=mask, cmap="coolwarm", center=0, vmin=-1, vmax=1, square=True,
            linewidths=.3, cbar_kws={"shrink":.7}, ax=ax)
ax.set_title("Correlation matrix of the 30 features")
plt.tight_layout(); plt.savefig("figures/v4_corr_heatmap.png", dpi=150); plt.show()
hp = corr.where(np.triu(np.ones(corr.shape),
  1).astype(bool)).stack().sort_values(ascending=False)
print("Most correlated feature pairs:\n", hp.head(5).round(3))
print("\nNumber of pairs with |r| > 0.9:", int((hp.abs() > 0.9).sum()))

# ===== V5: Pairplot of top features (Stage 5: Represent) =====
pp_feats = ["mean radius","mean texture","mean concave points","worst perimeter"]
g = sns.pairplot(df[pp_feats+["diagnosis"]], hue="diagnosis", palette=PAL,
  corner=True,
                 plot_kws={"s":18,"alpha":.65}, diag_kind="kde", height=2.2)
g.fig.suptitle("Pairplot of four informative features", y=1.02)
g.savefig("figures/v5_pairplot.png", dpi=150); plt.show()

# ===== V6: Refined scatter with trend and annotations (Stage 6: Refine) =====
fig, ax = plt.subplots(figsize=(8,6))
sns.scatterplot(data=df, x="worst perimeter", y="worst concave points",
  hue="diagnosis", palette=PAL,
                s=45, alpha=.75, edgecolor="white", linewidth=.4, ax=ax)
ax.axvline(105, ls="--", c="grey", lw=1); ax.axhline(0.14, ls="--", c="grey", lw=1)
ax.text(106, 0.015, "perimeter ~105", fontsize=8, color="grey")
ax.text(52, 0.145, "concave pts ~0.14", fontsize=8, color="grey")
ax.set_title("Worst perimeter vs. worst concave points\n(two strongest single
  predictors, refined view)")
ax.legend(title="Diagnosis", loc="lower right")
plt.tight_layout(); plt.savefig("figures/v6_refined_scatter.png", dpi=150); plt.show()

# ===== V7: Interactive scatter (Stage 7: Interact) =====
fig = px.scatter(df, x="worst perimeter", y="worst concave points", color="diagnosis",
                 color_discrete_map=PAL, size="mean area", size_max=14, opacity=.7,
                 hover_data={f: ":.2f" for f in ["mean radius","mean texture","mean
  concavity","worst area"]},
                 title="Interactive: hover, zoom, and click legend to filter by
  diagnosis")
```

```
fig.update_layout(template="plotly_white", height=520)
fig.write_html("figures/v7_interactive_scatter.html")
fig.show()

# ===== Step 4: Model Building =====
corr_t = X_tr_c.apply(lambda c: np.corrcoef(c, ytr)[0,1]).abs()
mi = pd.Series(mutual_info_classif(Xtr, ytr, random_state=SEED), index=FEATURES)
rf = RandomForestClassifier(n_estimators=400, random_state=SEED).fit(Xtr, ytr)
rf_imp = pd.Series(rf.feature_importances_, index=FEATURES)

imp_df = pd.DataFrame({"|corr| with target": corr_t, "mutual info": mi, "RF
  importance": rf_imp})
rank = imp_df.rank(ascending=False).mean(axis=1).sort_values()
top = rank.head(10).index
display(imp_df.loc[top].round(3))

fig, axes = plt.subplots(1,3, figsize=(15,5))
for ax, col, c in zip(axes, imp_df.columns, ["#457b9d","#e9c46a","#2a9d8f"]):
    imp_df[col].sort_values().tail(10).plot.barh(ax=ax, color=c); ax.set_title(col)
plt.suptitle("Top-10 features by three importance measures", y=1.02)
plt.tight_layout(); plt.savefig("figures/f1_feature_importance.png", dpi=150,
  bbox_inches="tight"); plt.show()

# ===== 4.2 MLP architecture and hyperparameter tuning =====
def build_mlp(units=32, dropout=0.2, lr=1e-3, l2=1e-4):
    reg = keras.regularizers.l2(l2)
    m = keras.Sequential([
        layers.Input(shape=(Xtr.shape[1],)),
        layers.Dense(units, activation="relu", kernel_regularizer=reg),
        layers.BatchNormalization(), layers.Dropout(dropout),
        layers.Dense(units//2, activation="relu", kernel_regularizer=reg),
        layers.BatchNormalization(), layers.Dropout(dropout),
        layers.Dense(1, activation="sigmoid")])
    m.compile(optimizer=keras.optimizers.Adam(lr), loss="binary_crossentropy",
              metrics=["accuracy", keras.metrics.AUC(name="auc")])
    return m

def fit(m, epochs=150, verbose=0):
    es = keras.callbacks.EarlyStopping(monitor="val_loss", patience=15,
  restore_best_weights=True)
    return m.fit(Xtr, ytr, validation_data=(Xva, yva), epochs=epochs, batch_size=32,
  callbacks=[es], verbose=verbose)

import itertools
grid = list(itertools.product([16, 32, 64], [0.1, 0.3], [1e-3, 5e-3]))
rows = []
for u, d, lr in grid:
    keras.utils.set_random_seed(SEED)
    m = build_mlp(u, d, lr); h = fit(m)
    vl = min(h.history["val_loss"]); i = int(np.argmin(h.history["val_loss"]))
    rows.append(dict(units=u, dropout=d, lr=lr, val_loss=vl,
  val_acc=h.history["val_accuracy"][i],
                     val_auc=h.history["val_auc"][i], epochs=i+1))
tune = pd.DataFrame(rows).sort_values(["val_loss"]).reset_index(drop=True)
display(tune.round(4))
best = tune.iloc[0]; print("Best config:", dict(units=int(best.units),
  dropout=best.dropout, lr=best.lr))
```

```
keras.utils.set_random_seed(SEED)
model = build_mlp(int(best.units), float(best.dropout), float(best.lr))
model.summary()
history = fit(model)
print("Stopped after", len(history.history["loss"]), "epochs")

# ===== 4.3 Permutation importance on the trained MLP =====
def perm_importance(model, X, y, n_rep=15, seed=SEED):
    rng = np.random.default_rng(seed)
    base = accuracy_score(y, (model.predict(X, verbose=0).ravel() > .5).astype(int))
    out = {}
    for j, f in enumerate(FEATURES):
        drops = []
        for _ in range(n_rep):
            Xp = X.copy(); Xp[:, j] = rng.permutation(Xp[:, j])
            drops.append(base - accuracy_score(y, (model.predict(Xp,
  verbose=0).ravel() > .5).astype(int)))
        out[f] = np.mean(drops)
    return pd.Series(out).sort_values(ascending=False)

pi = perm_importance(model, Xva, yva)   # computed on validation set
fig, ax = plt.subplots(figsize=(7,5))
pi.head(10)[::-1].plot.barh(color="#6a4c93", ax=ax)
ax.set_title("MLP permutation importance (accuracy drop, validation)");
  ax.set_xlabel("Mean accuracy drop")
plt.tight_layout(); plt.savefig("figures/f2_perm_importance.png", dpi=150); plt.show()
print(pi.head(8).round(4))

# ===== Step 5: Model Evaluation Visualization =====
p_te = model.predict(Xte, verbose=0).ravel(); yhat = (p_te > .5).astype(int)
lr_base = LogisticRegression(max_iter=2000).fit(Xtr, ytr)
p_lr = lr_base.predict_proba(Xte)[:,1]

def metrics(y, p, name):
    yh = (p > .5).astype(int); fpr, tpr, _ = roc_curve(y, p)
    return dict(model=name, accuracy=accuracy_score(y,yh),
  precision=precision_score(y,yh),
                recall=recall_score(y,yh), f1=f1_score(y,yh), auc=auc(fpr,tpr))
res = pd.DataFrame([metrics(yte, p_te, "MLP (Keras)"), metrics(yte, p_lr, "Logistic
  Regression")]).set_index("model")
display(res.round(4))
print(classification_report(yte, yhat, target_names=["Benign","Malignant"], digits=3))

# ===== E1: Loss vs. epoch =====
h = history.history; ep = np.arange(1, len(h["loss"])+1)
fig, ax = plt.subplots(figsize=(7,4.5))
ax.plot(ep, h["loss"], label="Train loss", c="#e76f51"); ax.plot(ep, h["val_loss"],
  label="Validation loss", c="#264653")
ax.axvline(int(np.argmin(h["val_loss"]))+1, ls="--", c="grey", label="Best epoch")
ax.set_xlabel("Epoch"); ax.set_ylabel("Binary cross-entropy"); ax.set_title("Loss vs.
  epoch"); ax.legend()
plt.tight_layout(); plt.savefig("figures/e1_loss.png", dpi=150); plt.show()

# ===== E2: Accuracy vs. epoch =====
fig, ax = plt.subplots(figsize=(7,4.5))
ax.plot(ep, h["accuracy"], label="Train accuracy", c="#e76f51"); ax.plot(ep,
  h["val_accuracy"], label="Validation accuracy", c="#264653")
```

```
ax.set_xlabel("Epoch"); ax.set_ylabel("Accuracy"); ax.set_ylim(0.85,1.005);
  ax.set_title("Accuracy vs. epoch"); ax.legend()
plt.tight_layout(); plt.savefig("figures/e2_accuracy.png", dpi=150); plt.show()

# ===== E3: Confusion matrix =====
cm = confusion_matrix(yte, yhat)
fig, ax = plt.subplots(figsize=(5,4.5))
ConfusionMatrixDisplay(cm, display_labels=["Benign","Malignant"]).plot(cmap="Blues",
  ax=ax, colorbar=False)
ax.set_title("Confusion matrix (test set)")
plt.tight_layout(); plt.savefig("figures/e3_confusion.png", dpi=150); plt.show()
tn, fp, fn, tp = cm.ravel(); print(f"TN={tn} FP={fp} FN={fn} TP={tp}")

# ===== E4: ROC curve and AUC =====
fig, ax = plt.subplots(figsize=(6,5.2))
for p, name, c in [(p_te,"MLP","#e76f51"), (p_lr,"Logistic Regression","#264653")]:
    fpr, tpr, _ = roc_curve(yte, p); ax.plot(fpr, tpr, label=f"{name} (AUC =
  {auc(fpr,tpr):.3f})", c=c, lw=2)
ax.plot([0,1],[0,1],"k--",lw=1,label="Chance"); ax.set_xlabel("False positive rate");
  ax.set_ylabel("True positive rate")
ax.set_title("ROC curve (test set)"); ax.legend(loc="lower right")
plt.tight_layout(); plt.savefig("figures/e4_roc.png", dpi=150); plt.show()

# ===== E5: Prediction-error distribution (classification analogue of an error plot)
  =====
fig, axes = plt.subplots(1,2, figsize=(12,4.5))
for lab, name in [(0,"Benign"),(1,"Malignant")]:
    sns.histplot(p_te[yte==lab], bins=20, color=PAL[name], alpha=.7, label=f"True
  {name}", ax=axes[0])
axes[0].axvline(.5, c="k", ls="--"); axes[0].set_xlabel("Predicted P(malignant)");
  axes[0].set_title("Predicted probability by true class"); axes[0].legend()
err = np.abs(yte - p_te)
sns.histplot(err, bins=20, color="#6a4c93", ax=axes[1]); axes[1].set_title("Absolute
  error |y - p|"); axes[1].set_xlabel("Absolute error")
plt.tight_layout(); plt.savefig("figures/e5_error_distribution.png", dpi=150);
  plt.show()
print("Mean absolute error:", err.mean().round(4), "| samples with error > 0.5:",
  int((err>.5).sum()))
```